

Unity Client Programmer

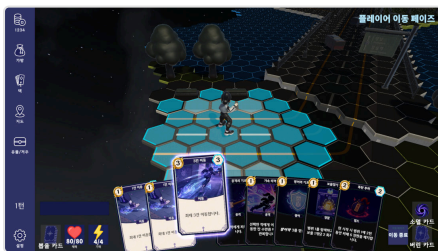
이예빈

YEBIN LEE

게임 규칙을 엔진에서 떼어내고,
성능은 추측이 아니라 계측으로 다룹니다.

턴제 턱빌딩 로그라이크 기억결(MemoryStone)의 클라이언트를 전담 개발하고 있습니다. 전투·맵·카드 규칙을 UnityEngine 비의존 어셈블리로 분리해 싼 없이 검증되게 만들었고, Profiler로 병목을 특정해 턴당 비용을 10배, 싼 로드를 31배 줄였습니다.

Email	puding2564@gmail.com
GitHub	github.com/binnini
Phone	010-9473-7840
Portfolio	binnini.github.io/portfolio



기억결(2026) · Rainbow War(2022) · 설화(2022)

Skills

Language	C# · HLSL · Python	성능	Unity Profiler · ProfilerMarker / ProfilerRecorder · Frame Debugger · 배칭·드로우콜 예산 관리
Engine	Unity 6 (6000.3) · URP · Input System · Cinemachine · Timeline	품질	EditMode / PlayMode 자동화 테스트 · 골든 베이 스라인 회귀 게이트
설계	Assembly Definition 계층 분리 · 데이터 주도 설계 · 상태 머신 · 의존성 역전	협업	UVCS(Plastic SCM) · Git · 스펙/결정 기록 · AI 코딩 도구(Claude Code, Codex, Unity MCP)

Projects

기억결 / MemoryStone 팀 리더 · 개발 전담 (3인)

2026.06 ~ 진행 중

육각 타일 던져 데빌딩 로그라이크. 게임 로직 전체와 Unity 클라이언트를 구현. C# 509파일 · 약 15만 줄 / 자동화 테스트 218파일. 제1회 서울 플레이업 AI 게임 챌린지 상위 8팀 선정, 2026 GES 전시·발표 예정.

Rainbow War 개발 리더 (7인)

2021.04 ~ 2022.04

실시간 멀티플레이 영역 점령 TPS, Photon 기반 상태 동기화와 Ear Clipping 삼각분할 영역 생성 알고리즘 담당. 아주대 교내 동아리 시연회 출품.

설화 개발 전담 (4인)

2022.04 ~ 2022.12

조선 배경 횡스크롤 던져 로그라이크. 캐릭터·스토리를 JSON 데이터로 관리해 콘텐츠 확장이 쉬운 구조를 만들고, Observer·Factory 패턴으로 전투 로직을 정리. 소학회 시연.

Match & Mist 개발 참여 (Global Game Jam)

2022 · 3일

제한 시야 추격 프로토타입. 이동 속도가 소음이 되고 좀비가 소음을 추적하는 규칙을 구현. 게임잼 3일 안에 플레이 가능한 빌드를 완성해 제출.

이 문서에서 다루는 것

#	주제	핵심 결과
01	엔진 비의존 코어와 Unity 어댑터 분리	썬 없이 도는 EditMode 검증 체계
02	축좌표 기반 육각 그리드와 경로 탐색	조건 분기 없는 거리·회전·방향 판정
03	턴 상태 머신과 적 행동 예고	“프리뷰가 곧 실제 결과”를 구조로 보장
04	암시야 — 렌더링이 아닌 정보 은닉 계층	필터를 우회할 수 없는 데이터 경계
05	계층 기반 성능 최적화	턴당 10배 · 썬 로드 31배
06	회귀 게이트와 대규모 리팩토링	10,690줄 God Object 분해

서울의 랜드마크와 전통 요괴를 배경으로 한 육각 타일 턴제 덱빌딩 로그라이크입니다. 제한된 시야 속에서 카드로 이동·공격·정찰을 하며 서울을 탐험합니다. 3인 팀에서 팀 리더 겸 개발 전담으로 게임 로직 전체와 클라이언트를 구현했습니다.



메인 게임플레이 — 육각 타일 맵, 제한 시야, 카드 레인, 몬스터 행동 예고



역할

팀 일같이 게임즈 · 3인 (개발/기획 · 기획 보조 · 아트)
 담당 게임 로직 · 클라이언트 전담, 기획 · UI/UX, 셰이더, 아트 방향 조
 울
 기간 2026.06 ~ 진행 중 (공모전 제작 1개월 후 계속 개발)
 성과 제1회 서울 플레이업 AI 게임 챌린지 상위 8팀 · 2026 GES 전시·
 발표 예정

규모

제품 코드	509 파일 / 약 149,500줄
어셈블리	제품 11 · 에디터 3 · 테스트 5
자동화 테스트	218 파일 · EditMode 925 pass
직접 작성 셰이더	11종
ScriptableObject	29종
버전 관리	UVCS (Plastic SCM)

이 프로젝트에서 스스로 세운 원칙 — 사람의 기억과 주의력에 의존하지 않는 구조를 만든다. 규칙은 당부가 아니라 컴파일러가 강제하게 하고(01), 성능은 감이 아니라 게이트가 지키게 합니다(06).

전투·맵·카드 규칙을 **UnityEngine**을 참조할 수 없는 어셈블리로 떼어내고, 그 경계를 컴파일러가 강제하게 했습니다.

문제

규칙이 MonoBehaviour 안에 있으면 “공격 카드가 맞게 처리되는가”를 확인하려고 매번 Unity를 켜고 Play를 눌러야 합니다. 턴제 게임은 규칙이 계속 바뀌는데, 손으로 플레이해서 확인하는 방식으로는 **A를 고쳤더니 B가 깨진 것**을 잡을 수 없습니다.

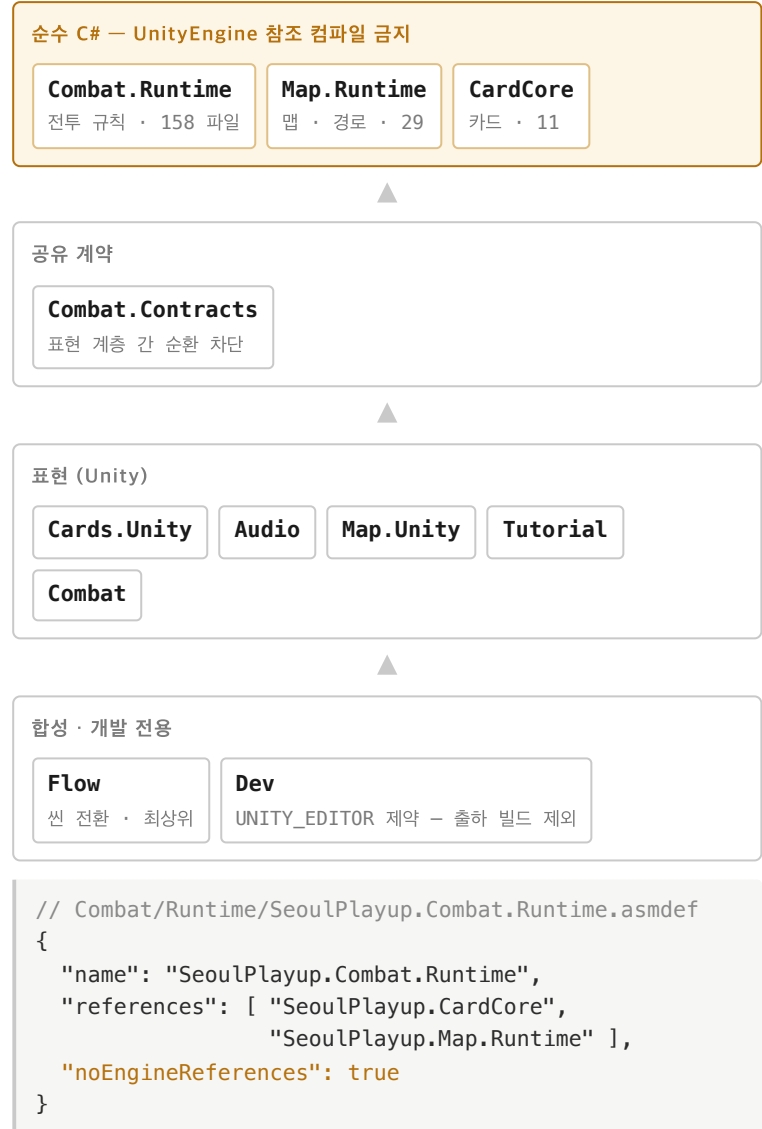
접근

- 의존 방향을 **표현** → 런타임 단방향으로 고정
- 순수 계층 3개에 `noEngineReferences: true`
— `using UnityEngine` 이 컴파일 에러
- 표현 계층이 전투 상태를 읽어야 하는 자리는 역참조 대신 seam으로 해결

순환을 끊은 세 가지 수법

인터페이스 인버전	<code>ICombatAudioHost</code>
델리게이트 인버전	<code>LabelFontApplier</code>
공유 계약 어셈블리	<code>Combat.Contracts</code>

어셈블리 참조 그래프



Unity에서 특히 신경 쓴 것 — 어셈블리를 쪼개면 `internal` 접근이 끊깁니다. 분리 전 접근성을 보존해야 하는 곳만 `InternalsVisibleTo`로 열되, 새 코드가 거기 기대는 설계는 하지 않는다는 규칙을 함께 정했습니다. 그러지 않으면 분리한 의미가 사라지기 때문입니다.

결과 — 전투·맵·카드 규칙이 **씬 없이 EditMode에서 검증**됩니다. 현재 참조 그래프 위반 0건, EditMode 스위트 925 pass. `Combat.Runtime.dll`은 Unity 없이도 로드해 실행할 수 있습니다.

육각 격자를 3차원 큐브 좌표의 대각 평면으로 다뤄, 거리·회전·방향 판정에서 행 흘쩍 분기를 완전히 없앴습니다.



GetReachableCells 가 반환한 이동 가능 범위와 선택 경로

문제

육각 격자는 이웃이 6개인데 **행마다 반 칸씩 어긋나**다. (행, 열) 로 저장하면 짝수 행과 홀수 행의 이웃 계산식이 서로 달라지고, 거리 계산은 더 복잡해집니다.

접근

축을 3개 두고 $q + r + s = 0$ 제약을 겁니다. 저장은 q, r 두 개만 하고 s 는 파생시켜 **불일치를 원천 차단**했습니다.

```
// 저장은 2개, 사용은 3축. S는 제약에서 유도된다.
public int S => -Q - R;

// 거리 - 한 칸 이동 시 합이 0이어야 하므로
// 항상 두 축이 ±1씩 변한다. 그래서 2로 나눈다.
public int DistanceTo(HexCoord o)
    => (Abs(Q-o.Q) + Abs(R-o.R) + Abs(S-o.S)) / 2;

// 60도 회전 = 좌표 순환 한 번. 삼각함수 불필요.
// 공격 패턴을 6방향으로 돌릴 때 사용.
var nextQ = -r;
var nextR = q + r;

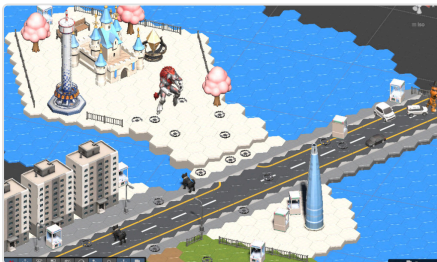
// 방향 판정 - 큐브 공간 내적 최댓값.
// 동점은 enum 순서로 갠다(결정성 확보).
var dot = dq*d.Q + dr*d.R + ds*(-d.Q-d.R);
```

경로 탐색 — HexPathfinder

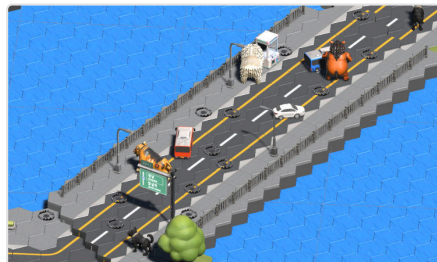
- 지형별 BaseMoveCost 가 달라 단순 BFS가 아닌 **가중 최단경로**
- GetReachableCells (범위 하이라이트)와 FindPath (실제 경로)가 같은 계산을 공유해 **규칙이 어긋날 수 없음**
- 맵·런타임 점유·지형 특성을 전부 **인자로 주입** — 숨은 상태가 없어 어떤 상황도 테스트 가능

확장 사례 · 멀티셀 보스의 통과 판정

여러 칸을 차지하는 보스가 자기가 지나갈 수 없는 틈으로 걸어 들어가는 문제가 있었습니다. 처음에는 보스 이동을 아예 막아줬다가, MovementQuery.FootprintRadius 를 더해 “중심 칸 하나”가 아니라 **중심 기준 반경 R 원판 전체**가 통과 가능한지를 묻도록 바꿨습니다. 기본값 0이면 기존 동작 그대로라 **기존 호출부를 한 곳도 고치지 않았고**, 자기 몸이 이미 깔고 있는 칸은 점유 판정이 OccupyingUnitId != UnitId 인 덕분에 **예외 처리 없이 자동으로 통과**합니다.



석촌호수 — 실제 서울 지형을 육각 타일로 재구성



잠실대교 — 이동 비용이 다른 지형이 섞인 구간



필드 카드가 만드는 오브젝트도 같은 좌표계 위에 놓인다

결과 — 좌표계를 바꾸자 거리·회전·방향이 각각 한 줄로 정리됐고, 이동·시야·공격 패턴이 모두 같은 좌표 연산 위에서 돌아갑니다. 전부 Map.Runtime (순수 C#)에 있어 씬 없이 검증됩니다.

적이 다음 턴 행동을 미리 보여주는 게임에서 생명은 “예고가 반드시 지켜진다”는 신뢰입니다. 그 신뢰를 규율이 아니라 **자료 구조로 강제**했습니다.



몬스터가 다음 턴 이동 위치와 공격 범위를 미리 노출한다

페이지 순환

```
PlayerMovement  -EndAction()→
MonsterMovement -Resolve→
PlayerAction     -EndAction()→
MonsterAction    -Resolve→ (다음 턴)
```

이동만으로는 페이지가 전환되지 않습니다.

EndAction() 이 유일한 전환 트리거입니다. 전환 경로가 여러 개면 조합이 폭발하고 그중 하나의 이상 상태를 놓치게 되는데, 입구를 하나로 두면 “왜 넘어갔지?”의 답이 항상 하나입니다.

순서가 곧 게임 규칙이다

턴 시작 시퀀스를 계약으로 고정했습니다 — ① 블록(방어도) 해제 → ② 필드 오브젝트 틱 → ③ 지속 효과 → ④ 예산·시야·인텐트 갱신 → ⑤ 페이지즈 진입. **따라서 필드 피해는 블록이 해제된 뒤에 들어옵니다.** ①과 ②를 뒤집으면 “방어도를 쌓아두면 다음 턴 장판을 막을 수 있는” 완전히 다른 게임이 되기 때문에, 문서로 고정하고 테스트로 잠갔습니다.



이동 예고와 공격 범위가 칸 집합으로 표시된다



여러 몬스터의 목적지가 겹치지 않도록 계획 시점에 예약된다

예고를 지키게 만든 세 장치

- **커밋 시점 좌표 보존** — `EnemyIntent.EnemyCoord` 는 라이브 좌표가 아니라 예고를 만든 시점의 좌표를 복사해 알려줍니다. 몬스터가 움직여도 예고가 따라가지 않습니다.
- **전 구간 잠금** — 커밋된 인텐트는 플레이어의 이동·액션·턴 종료 전 구간에서 유지됩니다. 플레이어가 움직였다고 다시 조준하지 않습니다.
- **회피의 정의를 좁힘** — 공격 범위를 규칙이 아니라 **구체적인 칸의 집합**으로 확정해 보여줍니다. 집합 밖으로 나가야 회피이고, 안에서의 재배치는 회피가 아닙니다.

플랜 예약 — 프리뷰가 곧 플랜

몬스터 둘에게 각각 “가장 가까운 칸으로”라고 하면 **둘 다 같은 칸을 고르고** 해석 단계에서 하나가 밀려납니다. 그러면 예고와 결과가 달라집니다.

계획 시점에 목적지를 **예약**해 서로 겹치지 않게 했습니다. 예고를 만드는 계산과 실제 이동을 해석하는 계산이 **같은 결과를 내도록 보장**합니다.



현재 페이지와 남은 기력을 항상 노출

결과 — 페이지 순환·인텐트 커밋·회피 판정·목적지 예약이 모두 EditMode 테스트로 잠겨 있어, 새 카드나 상태이상을 추가해도 “예고 = 실제 결과”가 깨지면 즉시 빨간불이 켜집니다.

안개를 “안 보이는 칸을 검게 칠하는 일”이 아니라 “렌더러가 원본 데이터를 아예 받지 못하게 하는 일”로 설계했습니다.



정찰 카드로 앞을 밝힌다. 시야는 순수 육각 거리로만 계산된다

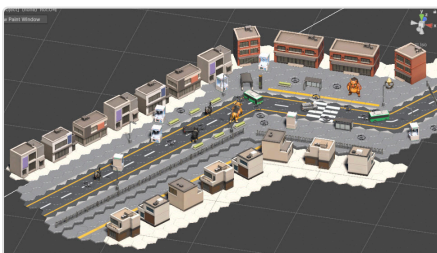
문제

“검게 칠하기”로 만들면 렌더러는 원본 데이터를 전부 들고 있으면서 화면에만 안 그립니다. 그러면 툴팁이 안개 속 칸 이름을 띄우고, 미니맵이 아직 못 본 랜드마크를 표시하고, 새 UI를 만든 사람이 필터를 빠뜨립니다. 표면이 늘 때마다 누출 가능성이 생깁니다.

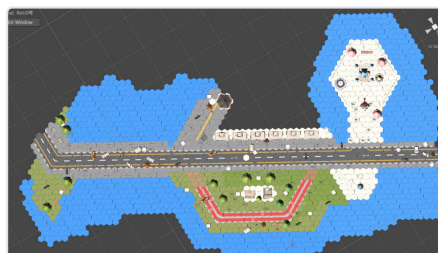
단계별 노출 경계

노출 항목	Hinted	Revealed
지형 · 이동비용 · 보행 · 높이	○	○
타일 정의 · 이벤트 · 랜드마크	×	○

Hinted는 “지나갈 수 있는 땅인가”까지만 답하고 “거기 무엇이 있는지”는 숨깁니다. 플레이 경험으로는 “길이 있는 건 알겠는데 뭐가 있는진 모르겠다”가 됩니다.



탐험 초반 — 주변만 Revealed, 나머지는 Unknown



진행 후 — 지나온 칸은 Hinted로 남고 지형만 보인다



툴팁도 같은 필터를 거친 뷰만 읽는다

해결 — 걸러진 뷰만 건넨다

```
// HexVisibilityRuntime.GetSafeCellInfo
if (visibility == Unknown)
    return SafeCellInfo.Unknown(coord); // 아무것도 안 줌

if (visibility == Hinted)
    return new HexVisibilitySafeCellInfo(
        coord, visibility, true, true, false,
        string.Empty, // 타일 정의 id 숨김
        cell.TerrainTypeId, // 지형 노출
        cell.BaseMoveCost, // 이동 비용 노출
        cell.BaseWalkable, // 보행 가능 노출
        cell.BaseBlocksVision,
        string.Empty, // 이벤트 id 숨김
        string.Empty, // 랜드마크 id 숨김
        cell.VisualFloor, isTrapRevealed);

return new HexVisibilitySafeCellInfo( /* 전부 노출 */ );
```

원본 cell은 이 함수 밖으로 나가지 않습니다. 나가는 것은 단계에 따라 걸러진 별도 구조체뿐이라 소비자가 필터를 우회할 방법이 아예 없습니다. 새 UI를 만드는 사람이 실수하려면 할 수가 없습니다.

상태 규칙

- 승급은 단조 — 한 번 밝혀진 칸을 낮추려는 시도는 무시됩니다
- 임시 공개 = (플레이어 시야 U 유효한 필드 오브젝트 U 이번 턴 정찰)의 합집합을 매 갱신마다 재구성 — 차감식이면 소스끼리 서로를 지우는 버그가 납니다
- 저장 복원 시 먼저 전부 지우고 복원 — 단조 승급 때문에 덮어쓰면 전투 시작 시의 공개가 섞여 들어옵니다

설계 → 구현 → 최적화까지 — 이 암시야를 화면에 그리는 MapVisibilityLit.shader를 직접 작성했고(전체 11종 중 하나), 이후 그 셰이더의 조명 마스크가 프레임 비용의 병목으로 측정돼 다음 페이지의 최적화 대상이 됩니다.

네 건 중 세 건이 “더 빠르게”가 아니라 “아예 하지 않게” 만든 최적화였습니다. 10배·31배 같은 숫자는 상수 개선이 아니라 작업 자체를 제거해야 나옵니다.

기법	대상	전	후	개선
더티 플래그 증분 갱신	<code>Vis.ApplyVisibility</code>	59 ms/turn	5.8 ms	10×
입력 해시 기반 캐시	<code>Vis.OverlayBuild</code>	10.4 ms/move	2.1 ms	4.9×
사전계산 LUT	<code>Atlas.Vis.LightingMask</code>	1.97 ms/frame	0.18 ms	10.8×
씬 직렬화 데이터 감축	MainGameplay 씬 로드	17,480 ms	561 ms	31×

진단 — CPU인가 GPU인가를 먼저 갈랐다

```
steady    : 8.8 ms   draw call 205
move_turn : 56.0 ms  draw call 210
                ↳ 그릴 양은 그대로, 시간만 6배
```

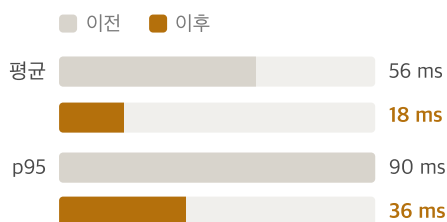
드로우콜이 늘지 않았으니 렌더링이 아니라 턴 처리 C# 코드가 원인입니다. 이 갈림길을 건너뛰면 “느리니까 배칭을 늘리자”로 잘못 갑니다.

특징 — ProfilerMarker를 계층으로 심었다

```
RefreshVisibilityHighlights    64 ms
├─ Vis.ApplyVisibility         59 ms ← 범인
├─ Vis.OverlayBuild            4 ms
│   └─ Vis.OverlayApply        1.2 ms
Pathfind                      4.8 ms
RefreshHud                    0.01 ms
```

ProfilerRecorder로 이 수치를 코드에서 읽어 테스트가 CSV로 자동 기록합니다. 눈으로 보는 프로파일링은 재현도 기록도 안 되기 때문입니다.

턴당 프레임 비용 (move_turn, ms)



60fps 예산이 16.7 ms입니다. 이전에는 한 칸 움직일 때마다 예산을 3~5배 넘겨 이동할 때마다 화면이 걸렸습니다. 사람은 평균이 아니라 최악을 체감하므로 p95를 함께 봅니다.

대표 사례 — 증분 시야 갱신

몇 칸이 바뀌었든 매 갱신마다 맵 전체의 안개 메시지를 파괴하고 다시 만들고 있었습니다. 게다가 갱신이 턴당 두 번 돌았습니다.

- 셀 단위 더티 — 각 칸의 (Exists, Visibility)를 기억해 두고 직전과 달라진 칸만 재적용
- 버전 스탬프 얼리 아웃 — 호출자가 “바뀐 것 없음”을 알려주면 맵 전체 재스캔을 통째로 건너뛴. 선택만 바꾸는 갱신은 바뀐 칸이 0이므로 낭비되던 두 번째 패스가 함께 사라졌습니다

다만 카운터와 체크 멤버십은 일부러 전 셀에 대해 계속 계산합니다(저렴하므로). 공개 계약을 바꾸지 않기 위해서입니다 — 빨라졌지만 겉보기 결과는 완전히 동일합니다.

계측 도구 자체가 틀렸던 일

측정 하네스가 이동 페이지 제약 때문에 실제 이동을 한 번만 유발하고 있어 per-move 비용이 희석돼 있었습니다. 매 반복마다 인접 칸으로 이동하도록 고치자 진짜 병목(OverlayBuild 10.4 ms)이 드러났습니다.

되돌린 최적화

안개 체크 재생성(약 2.5 ms)을 줄이려고 Mesh-버퍼 재사용 풀을 넣었지만 측정상 차이가 없어 되돌렸습니다. GC 바운드가 아니라 compute 바운드였기 때문입니다. 객체를 자주 만드는 것처럼 보여도 원인이 다르면 처방이 듣지 않습니다.

최적화가 바꾸지 않은 것

카운터와 체크 멤버십은 저렴하므로 일부러 전 셀에 대해 계속 계산합니다. 공개 계약을 바꾸지 않기 위해서입니다. 최적화가 겉보기 동작을 바꾸면 그건 개선이 아니라 회귀입니다.

결과 — 턴 이동 시 프레임 56 → 18 ms(p95 90 → 36 ms), 씬 로드 17.5초 → 0.56초. 개선은 전부 회귀 게이트로 잠가 되돌아가지 않게 했습니다(다음 페이지).

버그는 터지면 보이지만 **성능 회귀는 조용히 옵니다**. 그래서 사람의 주의력이 아니라 기계가 감시하게 만들었습니다.

무엇을 게이트로 삼을 것인가

프레임 시간은 기계마다 다릅니다. 이걸 게이트로 쓰면 코드가 멀쩡한데 빨간불이 뜨고, 곧 **사람들이 그 테스트를 무시하기 시작합니다**. 무시당하는 게이트는 없는 것보다 나쁩니다.

지표	성질	운동
renderer · 정점 · 삼각형 수	결정론적	정확히 일치 게이트
프레임 시간 · ms 비용	기계 의존	추세 기록만

골든 베이스라인

셀 {1,000·5,000·10,000} × 프롭 {25·100·250} × 배칭 {on·off} = **18개 조합**의 렌더 복잡도를 커밋된 JSON과 대조합니다. 파일이 없어도 실패하는 fail-closed 게이트입니다.

최적화가 “살아있음”을 증명하는 테스트

```
ChunkingReducesRenderersWithoutChangingGeometry
배칭 OFF / ON 을 각각 렌더한 뒤
· renderer 가 10배 이상 줄었는가? (작동 중인가)
· 정점·삼각형은 완전히 동일한가? (내용이 같은가)
```

배치는 조건이 어긋나면 **아무 에러 없이 폴백해** 조용히 느려집니다. 테스트 픽처를 일부러 조건에 맞춰 실제 배칭 경로를 타게 만들어, “적용했다”가 아니라 “**지금 작동 중이다**”를 보증합니다.

God Object 분해

MapCombatController.cs 가 **10,690줄 단일 클래스**까지 커졌습니다 (멤버 선언 766건, [SerializeField] 148건). 1개월 공모전 일정에서 기능을 우선했던 결과이고, 개발이 이어지면 서 값기 시작했습니다.

- 멤버 766건을 어느 seam에 속하는지 분류표부터 만들고 시작
- 가장 작은 seam(ICombatAudioHost , 멤버 6개)부터 단계별로
- 껍데기는 남기고 속만 옮기는 **strangler 방식**

Unity라서 지켜야 했던 제약

씬과 프리팹은 코드를 문자열·GUID로 참조합니다. 이름을 바꾸면 **컴파일은 되는데 실행하면 null**이 됩니다.

- [SerializeField] 148건은 **본체에 잔류** — 인스펙터 값과 참조 보존. 어댑터에는 값·델리게이트만 주입
- ICombatAudioHost 구현을 인터페이스 목록에서 제거하지 않음 — 씬의 MonoBehaviour를 자동 탐색하는 경로가 깨지기 때문

테스트가 이미 빨간불이었을 때

리팩토링 시작 시점의 EditMode 베이스라인은 **117개 실패** 상태였습니다. “전부 초록으로 만들고 시작”은 현실적으로 영원히 시작하지 못한다는 뜻이라, 게이트를 “**신규 실패 0**”으로 정의하고 단계마다 **실패 집합이 늘지 않았음**을 확인하며 전진했습니다.

씬 수술 — 재발 방지가 본체다

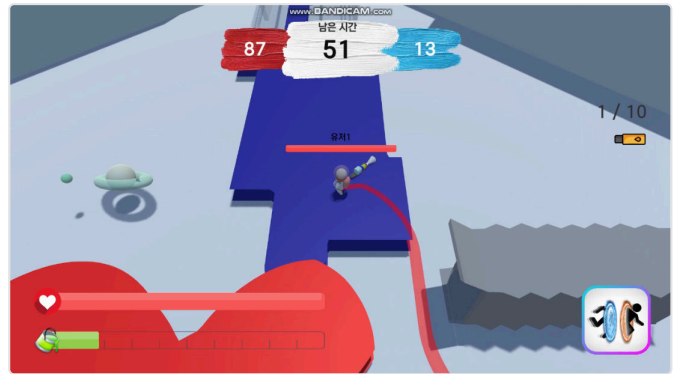
타일을 클릭하려고 콜라이더를 타일마다 붙여 두었더니 MainGameplay 씬에 **클릭 콜라이더 139,146개**가 구워져 씬 파일이 317 MB, 로드가 17.5초까지 갔습니다. 콜라이더 대신 스크린 좌표를 헥스 좌표로 직접 변환하는 방식(TryScreenToHex)으로 바꾸고 구워진 오브젝트를 전부 삭제해 **50 MB · 0.56초**가 됐습니다. 중요한 건 그다음입니다 — 골든 베이스라인에 clickColliders: 0 을 잠가 **콜라이더 생성 경로가 되 살아나면 게이트가 실패**하게 했고, 실제 씬·실제 카메라로 좌표 변환을 왕복 검증하는 테스트를 함께 넣었습니다.

결과 — 본체 10,690줄 → 2,233줄 + 관심사별 분할, 오디오·카메라는 별도 클래스로 추출. 베이스라인은 **117 실패에서 925 pass / 0 fail**로 초록이 됐습니다. 다만 CombatState.cs 는 아직 6,907줄로 남아 있어 분해가 진행 중입니다.

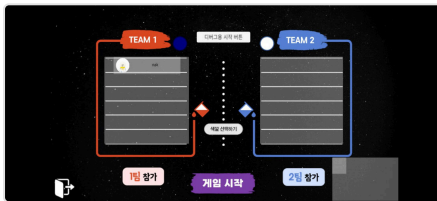
자기 진영에서 선을 그어 영역을 넓혀 가며 더 넓은 땅을 차지하는 실시간 멀티플레이 게임입니다. **개발 리더**로 팀을 이끌며 **상태 동기화와 영역 계산**을 담당했습니다.



선을 그어 영역을 확장하는 순간 — 선을 긋는 동안은 무방비 상태다



Ear Clipping으로 삼각분할해 생성한 영역 메시



담당 · 멀티플레이 상태 동기화

Photon RPC 기반으로 플레이어 이동·사격·피격과 영역 갱신을 동기화했습니다. 클라이언트마다 영역 계산 결과가 어긋나는 문제를 **Master-Slave 구조**로 정리해, 판정의 기준을 한 곳으로 모으고 나머지는 결과를 받아 반영하도록 했습니다.

담당 · 영역 생성 알고리즘

플레이어가 그린 선이 자기 영역과 이어지면 그 안쪽을 메워야 하는데, 임의의 다각형이라 그대로는 메시로 만들 수 없습니다.

Ear Clipping 삼각분할로 다각형을 삼각형으로 쪼개 영역 메시를 생성했습니다.

규칙 설계에서 배운 것

선을 긋는 도중 상대방에게 밟히거나 체력이 0이 되면 그리던 선이 취소되고 리스폰 동안 움직일 수 없습니다. 이 규칙 하나로 “무작정 크게 그리기”가 위험해지고 **사격으로 견제하는 이유**가 생겼습니다. 규칙 하나가 플레이 양상을 바꾼다는 걸 처음 체감한 프로젝트입니다.

결과

- 아주대 교내 동아리 시연회 출품, 현장에서 직접 시연
- 반복적인 플레이 검증의 중요성을 배움 — 코드상 맞는 동기화도 여러 명이 동시에 붙으면 다르게 깨진다
- 7인 팀의 개발 리더로 작업 분배와 통합을 담당

지금 돌아보면 — 당시에는 동기화 문제를 재현할 방법이 “여러 명이 모여서 해보기”뿐이었습니다. 코드상으로는 맞는데 네 명이 동시에 붙으면 영역 계산이 어긋나는 버그를, 사람을 모아 반복 재현하며 잡았습니다. 기억결에서 게임 로직을 엔진 비의존 계층으로 분리하고 자동화 테스트를 붙인 것은, 이때 겪은 “**재현할 수 없으면 고칠 수 없다**”는 문제의식에서 출발했습니다.

설화는 조선을 배경으로 한 횡스크롤 턴제 로그라이크로, 4인 팀에서 **개발을 전담**했습니다. Match & Mist는 Global Game Jam 2022에서 3일 만에 완성한 제한 시야 프로토타입입니다.



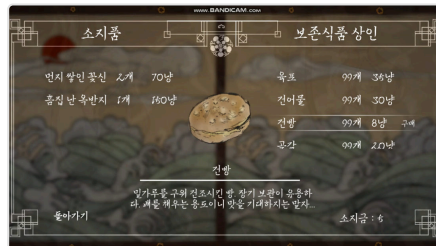
설화 — 보스전 진입 컷씬. 마을·상점·던전을 오가는 로그라이크 구조



Match & Mist — 성냥불로 안개를 밝히며 좀비를 피한다

설화 · 2022.04 ~ 2022.12 · 개발 전담

- 속도 순으로 턴이 도는 **파티 단위 턴제 전투**, 캐릭터별 고유 기술과 상태이상 구현
- 캐릭터·스토리·이벤트를 **JSON 데이터로 분리**해, 기획자가 콘텐츠 추가할 때 코드를 고치지 않아도 되는 구조를 만듦
- 전투 이벤트 전파에 Observer, 몬스터·아이템 생성에 Factory 패턴을 적용해 **8개월간 늘어나는 요구사항을 감당**
- 구미호·이무기 보스전에 별도 컷씬을 넣어 분위기를 끌어올림
- 소학회 내 시연 — 긴 협업에서 **요구사항 정리와 폴리싱의 중요성**을 배움



설화 — 던전 출발 전 마을에서 상점·게시판으로 준비를 갖춘다 · Match & Mist — 소음을 따라오는 좀비

Match & Mist · Global Game Jam 2022 · 3일

- 게임잼 주제 ‘**이중성**’에 맞춰, 팩맨처럼 아이템으로 **도망과 추격이 뒤바뀌는 순간**을 핵심 재미로 잡음
- 이동 속도가 빠를수록 소음이 커지고 좀비가 그 소음을 따라오는 규칙 — “**빨리 가고 싶은데 그러면 들킨다**”는 긴장을 만듦
- 성냥으로 잠시 시야를 밝히고, 칼로 잠시 반격할 수 있게 해 아이템에 역할을 분리
- 3일 안에 플레이 가능한 빌드를 완성해 제출 — **짧은 일정에서는 핵심 플레이 경험을 먼저 세워야 한다**는 것을 배움



세 프로젝트가 이어진 지점 — 설화의 JSON 데이터 분리는 기억결에서 카드·몬스터·상태이상을 CSV로 저작하고 에디터 임포터로 ScriptableObject를 생성하는 데이터 파이프라인으로 이어졌습니다. Match & Mist의 제한 시야는 기억결 암시야 설계의 출발점이었고, Rainbow War에서 겪은 재현 불가능한 버그가 자동화 테스트를 붙이게 된 계기였습니다.

09 그 외 경험 · Profile

인턴 · 영상 분석 AI

(주)베스텔라랩 · 영상 분석 AI 부서

2025.09 ~ 2025.12

주차장 CCTV 환경에서 차량 번호판을 인식해 차량 위치를 제공하는 서비스의 LPR(번호판 인식) 모듈을 담당했습니다. 회전·왜곡·원근 변환 증강으로 실제 CCTV 각도 문제에 대응하고, 모델을 파인튜닝했습니다.

인식 정확도

약 97~97.9%

추론 시간 (ONNX 변환 + FP16 양자화)

2.59s → **1.18s**

도메인은 다르지만 **계측하고 병목을 특정해 개선하고 다시 재는** 방식은 기억결의 성능 작업(05)과 같습니다.

Profile

Education

2020.03 ~ 2026.02 아주대학교 소프트웨어학과 졸업
학점 4.06 / 4.5 (전공 3.98 / 4.5)

Certification

2025.09 정보처리기사
2025.06 ADsP 데이터분석 준전문가

Activities

2026 제1회 서울 플레이업 AI 게임 챌린지 — 기억결, 상위 8팀
2022 게임 개발 소학회 TML — 설화, 개발 전담
2022 Global Game Jam 2022 — Match & Mist
2021 게임 개발 소학회 브레인스툼 — Rainbow War, 개발 리더

Links

GitHub github.com/binnini
Portfolio binnini.github.io/portfolio
기억결 빌드 · 시연 영상 · 발표 자료 (별첨 링크)

일하는 방식

기록으로 남기고 강제한다

3인 팀에서 개발을 혼자 맡았고 기획이 계속 바뀌었습니다. “저번에 왜 이렇게 정했더라”를 반복하지 않으려고 결정을 배경·대안과 함께 기록하기 시작했고, 시스템 스펙의 각 조항에 담당 테스트를 붙여 **문서와 코드가 어긋나면 테스트가 깨지도록** 했습니다.

AI 도구를 쓰는 관점

AI 활용의 한계는 도구가 아니라 **코드베이스 구조가 정한다**고 봤습니다. 게임 규칙을 순수 C#으로 분리해 CLI에서 테스트가 도는 영역을 만들자, 에이전트가 고치고 → 돌리고 → 다시 고치는 **검증 루프를 스스로 닫을 수 있게** 됐습니다. 규칙은 프롬프트로 지시하는 대신 **asmdef 제약**으로 컴파일러가 강제하게 했습니다.

덜어낸 판단도 기록했다

- 시야 차폐(line-of-sight)는 **구현하지 않기로 결정**하고 테스트로 잡았습니다 — 시야는 순수 거리라는 계약을 고정
- 어셈블리를 더 잘게 쪼갰지 검토했다가 **분리하지 않기로 종결**
- Mesh 재사용 풀은 넣었다가 **효과가 없어 되돌렸습니다**

공모전 1개월 구간에는 기능을 우선해서 밀었고, 개발이 계속 이어지게 되자 계측하고 분해하고 테스트를 채웠습니다. **급할 땐 빠르게 밀고, 오래 갈 프로젝트가 되면 갇는**다는 기준으로 판단했습니다.

앞으로 — 기억결은 지금도 개발 중이고 2026 GES 전시를 준비하고 있습니다. 남은 과제도 알고 있습니다. 측정이 아직 에디터 플레이 모드에 머물러 있어 **실기기 프로파일링**이 필요하고, CombatState 분해와 **Addressables 기반 리소스 관리**는 아직 손대지 못했습니다. 다음에 배우고 싶은 것도 그쪽입니다.